APPASA Strategy Document

Stage 3 – Process: Data Processing, Transformation & Validation

APPASA: Process Stage

Introduction

I will use this **APPASA strategy document** to record my decisions and reflections as I work through **Stage 3: Process** of this project. This document will serve as a guide, helping me consider my responses and reflections throughout this stage of the data analysis process.

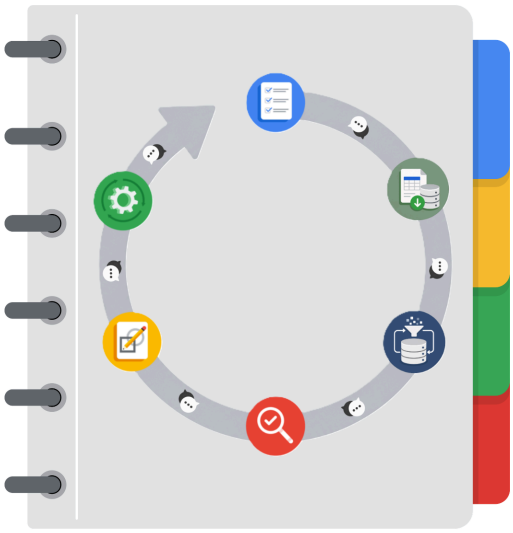
Portfolio Project Recap

Many of the goals I accomplished throughout the individual course portfolio projects are integrated into this end-to-end data analysis capstone project. These include:

- Creating a clear and structured project proposal.
- Demonstrating my understanding of SQL and relational database concepts.
- Using SQL to ingest, explore, clean, transform, validate, and organize data through structured queries and database operations.
- Organizing and analyzing a large-scale dataset to uncover meaningful insights and communicate the underlying business story.
- Developing a series of SQL scripts for ELT, feature engineering, data validation, and analytical querying, alongside a DuckDB-powered Jupyter Notebook documenting the complete APPASA methodology and reproducing the analytical workflow through SQL queries and visualizations.
- Computing descriptive statistics, performing exploratory and business-focused analyses, and creating analytical visualizations to support data interpretation and executive decision-making.
- Communicating findings through an executive summary, an interactive Power BI report, strategic business recommendations, and implementation-focused action plans for stakeholders.
- Translating analytical findings into evidence-based business recommendations, implementation strategies, success metrics, and future analytical opportunities through the final Act stage of the APPASA framework.

This capstone brings together the knowledge and technical skills I developed throughout the program, demonstrating my ability to execute a complete end-to-end data analysis project, from business problem definition to actionable business recommendations.

THE APPASA WORKFLOW



I will demonstrate to the company's marketing analytics team how I would apply the APPASA workflow to the upcoming Cyclistic Bike Share project. For each question presented in this APPASA strategy document, I will provide a structured and thoughtful response aligned with the corresponding stage of the APPASA framework: Ask, Prepare, Process, Analyze, Share and Act. This approach ensures clarity in my methodology, highlights my strategic planning skills, and illustrates a well-organized path from problem understanding to actionable insights.

Project Tasks

The following questions have been identified as essential to the Member vs Casual Rider Behavior Analysis Project. These questions are primarily situated within the Process stage of the APPASA workflow, focusing on validating data quality, cleaning and transforming the dataset, engineering analytical features, and ensuring the processed data is reliable and analysis ready. I will address each question by drawing on relevant materials from the SQL scripts, project notebook, the APPASA framework, and best practices in data processing and quality assurance.

Data Project Questions & Considerations

Data Processing Environment

- What tools were selected for data processing, and why?

The primary tool selected for data processing was **MySQL**, which served as the project's data processing engine. MySQL was chosen because it provides a robust relational database management system capable of efficiently handling millions of records while supporting SQL-based data ingestion, validation, cleaning, transformation, and feature engineering. Its support for Common Table Expressions (CTEs), window functions such as `ROW_NUMBER()`, indexing, and built-in date and string functions enabled the entire data preparation workflow to be performed within the database, creating a clean and reproducible ELT pipeline.

MySQL Workbench was used as the development environment for writing, executing, and documenting SQL scripts, inspecting intermediate results, and validating data quality throughout the processing workflow.

The raw datasets were initially stored as **CSV files**, preserving the original source data exactly as downloaded before being imported into MySQL. Maintaining the raw files separately from the processed database tables ensured data integrity, reproducibility, and an auditable processing pipeline. This separation also allowed the raw staging table to remain unchanged while a cleaned analytical table was created for downstream analysis.

Overall, this toolset supported an efficient, scalable, and well-documented data processing workflow that prepared the dataset for the subsequent Analyze stage.

- Why was MySQL chosen as the primary data processing environment?

MySQL was chosen as the primary data processing environment because it is specifically designed to efficiently manage and process large structured datasets using SQL. The Cyclistic dataset contained over **5.8 million trip records**, making a relational database management system more suitable than spreadsheet-based tools for scalable data processing.

MySQL enabled the entire processing workflow—including data ingestion, validation, cleaning, feature engineering, and data quality verification—to be performed within a single environment. Its support for Common Table Expressions (CTEs), window functions (`ROW_NUMBER()`), indexing, and built-in string, date, and conditional functions made it possible to implement a robust and reproducible ELT pipeline while preserving the raw dataset.

Using MySQL also separated the data processing stage from the analysis and visualization stages. Rather than repeatedly performing transformations within the business intelligence tool, a clean, processed analytical table was created once in the database and subsequently used for downstream analysis in Power BI. This approach improves maintainability, reproducibility, and processing efficiency while aligning with industry practices for managing large analytical datasets.

- How does the ELT workflow support this project?

The project follows an **Extract, Load, Transform (ELT)** workflow to preserve the integrity of the original data while creating a clean analytical dataset. First, the twelve monthly Divvy trip datasets were extracted from their source and loaded into a raw staging table within MySQL without modification. All subsequent data validation, cleaning, feature engineering, and quality verification were performed directly within the database, producing a separate processed table for downstream analysis.

This approach ensures that the original data remains unchanged, allowing every transformation to be reproduced, audited, or revised without re-importing the source files. It also separates raw and processed data, making the workflow more maintainable and reducing the risk of accidental data loss or modification. By performing transformations inside MySQL, the project leverages the database engine to efficiently process millions of records before the cleaned dataset is imported into Power BI for analysis and visualization.

Data Validation

- What validation checks were performed before cleaning?

Before applying any cleaning operations, a comprehensive data validation process was performed to assess the integrity, completeness, and consistency of the raw dataset. The validation phase began by verifying the dataset structure, including the total number of imported records and the table schema, to confirm that the data had been imported successfully.

The dataset was then examined for duplicate records by identifying duplicate `ride_id` values and manually investigating representative duplicate patterns to determine an appropriate deduplication strategy. Missing values were assessed by checking for both `NULL` values and blank strings in station names and station IDs. Destination coordinate fields were also validated to identify zero-valued latitude and longitude records representing missing spatial information.

Finally, temporal integrity checks were performed to detect records with invalid timestamps, including trips ending before they started, negative-duration trips, zero-duration trips, and trips exceeding 24 hours. These validation findings provided the evidence required to define the cleaning rules applied during the subsequent data cleaning stage.

- What data quality issues were identified?

The validation process identified several data quality issues that required remediation before analysis. Duplicate `ride_id` values were detected, consisting of both exact duplicate records and duplicate records with slight variations in the `ended_at` timestamp. Missing station information was also identified, with station names and station IDs represented as blank strings rather than standardized `NULL` values.

The destination coordinate fields contained records where both `end_lat` and `end_lng` were recorded as `0`, indicating missing geographic information instead of valid coordinates. Further investigation revealed that the majority of these records also exhibited temporal anomalies and were subsequently excluded during temporal cleaning, while the remaining valid records had their missing coordinates standardized to `NULL`.

Temporal validation also identified trips with invalid timestamps, including rides ending before they started, non-positive ride durations, and rides exceeding the project's maximum accepted duration of 24 hours. These issues informed the cleaning rules implemented to produce a reliable analytical dataset for downstream analysis.

- Why was it important to validate the raw dataset before cleaning?

Validating the raw dataset before cleaning ensured that all data quality issues were identified and understood before any modifications were made. Rather than applying generic cleaning operations, the validation phase provided evidence for each cleaning decision by quantifying issues such as duplicate records, missing values, invalid spatial data, and temporal anomalies.

This evidence-based approach helped prevent unnecessary or incorrect transformations while ensuring that every cleaning rule addressed a specific, verified data quality issue. It also preserved the integrity of the raw dataset by allowing the original data to remain unchanged throughout the investigation. As a result, the cleaning process became transparent, reproducible, and well-documented, with each transformation justified by the findings of the validation phase.

- How were duplicate records investigated before deciding on a cleaning rule?

Duplicate records were first identified by detecting repeated `ride_id` values within the raw dataset. Rather than immediately removing these records, representative duplicate groups were manually inspected to understand the nature of the duplication. This investigation revealed two distinct patterns: exact duplicate records, where all attribute values were identical, and timestamp-variant duplicates, where records shared the same `ride_id` but differed slightly in the `ended_at` timestamp.

Based on these findings, a deterministic deduplication strategy was established. The record with the most recent `ended_at` timestamp was retained for each `ride_id`, while all remaining duplicate records were excluded from the processed dataset. This approach ensured that every ride was represented by a single record while applying a transparent and reproducible retention rule supported by the validation findings.

Data Cleaning

- What cleaning rules were applied to the dataset??

The cleaning process applied a series of rule-based transformations to improve data quality while preserving the integrity of the original dataset. Duplicate records were resolved by retaining a single record for each `ride_id` based on the most recent `ended_at` timestamp. Blank station names and station IDs were standardized to `NULL` to ensure consistent representation of missing values.

Destination coordinates with values of `0` for both `end_lat` and `end_lng`, representing missing geographic information, were converted to `NULL`. Records with invalid temporal values were excluded from the processed dataset, including trips ending before they started and trips with durations exceeding the project's maximum accepted threshold of 24 hours.

These cleaning rules were applied during the transformation from the raw staging table to the processed analytical table, ensuring that the resulting dataset was consistent, reliable, and suitable for downstream analysis.

- Why were these cleaning rules selected?

The cleaning rules were selected based on the findings of the data validation phase rather than being applied as generic preprocessing steps. Each rule addressed a specific data quality issue identified during validation, ensuring that every transformation was evidence-based and justified. Duplicate records were resolved using a deterministic retention strategy, missing values were standardized to ensure consistent representation across the dataset, and invalid temporal records were excluded because they did not represent valid trips.

The objective was to improve the quality, consistency, and reliability of the dataset while preserving as much valid information as possible. By applying only validated and well-documented cleaning rules, the project minimized unnecessary data loss and produced a processed analytical dataset suitable for downstream analysis and visualization.

- Which records were removed rather than corrected?

Only records that could not be reliably corrected while preserving data integrity were excluded from the processed dataset. This included duplicate records retained under the project's deduplication strategy, where only one record was preserved for each `ride_id`, as well as records with invalid temporal values that did not represent valid trips. Specifically, trips ending before they started and trips exceeding the maximum accepted duration of 24 hours were removed from the processed dataset.

In contrast, issues that could be standardized without losing valid information were corrected rather than removed. For example, blank station names and station IDs were converted to `NULL`, and zero-valued destination coordinates representing missing geographic information were also standardized to `NULL`. This

approach minimized unnecessary data loss while ensuring the analytical dataset remained accurate, consistent, and reliable.

- How was the original raw dataset preserved?

The original raw dataset was preserved by following an Extract, Load, Transform (ELT) workflow. The twelve monthly Divvy trip datasets were first imported into a dedicated raw staging table within MySQL without applying any modifications. All subsequent validation, cleaning, feature engineering, and data quality verification were performed using SQL transformations that populated a separate processed analytical table, leaving the raw table unchanged throughout the project.

Maintaining separate raw and processed tables ensured that the original source data remained available for auditing, revalidation, or future processing if required. This separation also improved reproducibility by allowing the entire transformation pipeline to be rerun from the unchanged raw dataset whenever modifications to the cleaning or feature engineering logic were needed.

- Why was a separate processed table created?

A separate processed table was created to provide a clean, analysis-ready dataset while preserving the integrity of the original raw data. Rather than modifying the source records directly, all cleaning operations, feature engineering, and data quality improvements were applied during the transformation from the raw staging table to the processed analytical table.

This separation ensured that the processed analytical table served as the data source for downstream analysis and dashboard development in Power BI while leaving the raw dataset unchanged. It also improved maintainability and reproducibility by allowing the processing pipeline to be rerun whenever cleaning rules or engineered features required modification, with the raw dataset retained as a permanent reference.

Feature Engineering

- Which analytical features were engineered?

Three analytical features were engineered during the processing stage to support downstream analysis while maintaining a reusable analytical dataset:

• **`ride_length_minutes`** – Calculated as the exact ride duration in minutes from the difference between `started_at` and `ended_at`. This feature supports analyses involving trip duration, rider behavior, and ride length distributions.

• **`ride_date`** – Extracted from the `started_at` timestamp to provide a date-level attribute for temporal analysis and to establish a relationship with the Date dimension table in the Power BI data model.

• **`hour_of_day`** – Extracted from the `started_at` timestamp to represent the ride start hour, enabling analyses of hourly ride demand, peak usage periods, and rider behavior throughout the day.

These features were selected because they represent reusable ride-level attributes that can support a wide range of analytical tasks beyond the project's dashboard, while dashboard-specific classifications and aggregations were intentionally reserved for the Power BI semantic layer.

- Why were these features created?

The engineered features were created to enhance the analytical value of the processed dataset while maintaining a reusable and database-centric data model. Rather than requiring downstream tools to repeatedly derive common analytical attributes, these features were computed once during the processing stage and stored within the processed table.

Each feature represents an intrinsic property of an individual ride rather than a dashboard-specific calculation. Ride duration supports analyses of trip behavior, ride date enables temporal analysis and integration with the Power BI Date dimension, and ride start hour facilitates the analysis of hourly demand and usage patterns. By engineering these reusable attributes within MySQL, the project reduced redundant computations in downstream tools while producing a consistent analytical dataset suitable for a variety of reporting and analytical use cases.

- How do the engineered features support later analysis?

The engineered features provide reusable analytical attributes that simplify downstream analysis and reporting. By deriving these features during the processing stage, subsequent analyses can focus on identifying patterns and generating insights rather than repeatedly performing the same data transformations.

The `ride_length_minutes` feature enables analyses of trip duration, rider behavior, and ride length distributions. The `ride_date` feature supports date-based trend analysis and serves as the relationship key between the processed dataset and the Power BI Date dimension, enabling efficient time-based reporting. The `hour_of_day` feature facilitates analyses of hourly ride demand, peak usage periods, and rider activity throughout the day.

Engineering these features during processing also improves consistency by ensuring that all downstream analyses and visualizations use the same standardized definitions, reducing redundant calculations across analytical workflows.

- Why were these features created during processing rather than analysis?

These features were engineered during the processing stage because they represent reusable ride-level attributes derived directly from the raw data, rather than analytical or visualization-specific calculations. Computing them once during data processing ensures that all downstream analyses, reports, and dashboards use consistent feature definitions without repeatedly performing the same transformations.

Creating these features within MySQL also aligns with the project's ELT architecture by preparing a clean, analysis-ready dataset before it is imported into Power BI. This reduces redundant computations, improves processing efficiency, and allows the Analyze stage to focus on exploring patterns, answering business questions, and generating insights rather than performing fundamental data transformations.

Data Quality Verification

- How was the cleaned dataset verified?

The cleaned dataset was verified through a comprehensive post-cleaning data quality verification process to confirm that all cleaning rules had been successfully applied. Verification queries were executed to reconcile the raw and processed datasets by comparing record counts and quantifying the number of excluded records. Additional checks confirmed that duplicate `ride_id` values had been resolved, missing station fields were consistently represented as `NULL`, zero-valued destination coordinates had been standardized, and all remaining records satisfied the project's temporal integrity requirements.

These verification steps ensured that the processed dataset accurately reflected the intended cleaning rules and was internally consistent before proceeding to feature engineering and downstream analysis.

- How was feature engineering verified?

The engineered features were verified through a dedicated post-engineering verification process to ensure that each feature had been generated accurately and consistently across the processed dataset. Verification queries confirmed that the engineered columns contained no unexpected `NULL` values, that ride durations were correctly calculated from the original timestamps, that ride dates matched the date component of `started_at`, and that ride start hours accurately reflected the hour extracted from the original timestamp.

These validation checks ensured that the engineered features were complete, internally consistent, and correctly derived from the source data before being used for downstream analysis and dashboard development.

- How did you ensure the processed dataset was ready for analysis?

The processed dataset was prepared for analysis through a structured data processing workflow consisting of data validation, cleaning, feature engineering, and post-processing verification. Data validation identified and documented data quality issues before any transformations were applied. Cleaning rules were then implemented to resolve duplicate records, standardize missing values, and exclude invalid temporal records while preserving the original raw dataset. Reusable analytical features were subsequently engineered to enhance the dataset for downstream analysis.

Following these transformations, dedicated verification procedures confirmed that the cleaning rules and engineered features had been applied correctly and consistently. Together, these steps ensured that the processed dataset was accurate, internally consistent, and analysis-ready, providing a reliable foundation for the subsequent Analyze stage and dashboard development.

- How does the verification process improve confidence in the results?

The verification process improves confidence in the results by confirming that the intended data transformations were applied accurately and consistently throughout the processing pipeline. Rather than assuming the cleaning and feature engineering steps executed correctly, dedicated verification queries were used to validate the integrity of the processed dataset after each stage.

By confirming that data quality issues were successfully resolved, engineered features were correctly derived, and the processed dataset satisfied all defined quality requirements, the verification process reduced the risk of undetected errors propagating into subsequent analysis and visualization. This systematic validation increased the reliability, reproducibility, and overall credibility of the project's analytical findings.

Documentation & Reproducibility

- How was the processing workflow documented?

The processing workflow was documented through a structured and fully commented SQL script that records each stage of the data preparation pipeline. The script is organized into logical sections covering project setup, data ingestion, data validation, data cleaning, feature engineering, and data quality verification, with explanatory comments describing the purpose and rationale behind each step.

In addition to the SQL implementation, the processing methodology, validation findings, cleaning decisions, engineered features, and verification procedures were documented within the project's APPASA Process stage documentation. Together, these artifacts provide a transparent, reproducible record of the complete data processing workflow, enabling the project to be reviewed, understood, and replicated by other analysts.

- How does the SQL script support reproducibility?

The SQL script supports reproducibility by providing a structured, deterministic implementation of the entire data processing pipeline. Every stage—from data ingestion and validation to cleaning, feature engineering, and data quality verification—is explicitly defined using SQL statements that can be executed repeatedly to produce the same processed dataset from the same raw data.

Because all transformations are implemented within a single, well-documented script, the complete workflow can be rerun whenever the source data is updated or the processing logic is refined. This eliminates reliance on undocumented manual operations, ensures consistent application of cleaning and transformation rules, and enables other analysts to reproduce the processing workflow and obtain identical results under the same conditions.

- Why is documenting cleaning decisions important?

Documenting cleaning decisions is important because it provides a transparent record of how the raw data was transformed into the final analytical dataset. By clearly recording the rationale behind each cleaning rule, the project enables others to understand what changes were made, why they were necessary, and how they may influence subsequent analysis.

Well-documented cleaning decisions also improve reproducibility, maintainability, and collaboration. They allow the processing workflow to be reviewed, audited, or modified in the future while ensuring that transformations remain consistent and evidence-based. This documentation helps build confidence in the integrity of the processed dataset and supports the credibility of the project's analytical findings.